Sveučilište Josipa Jurja Strossmayera u Osijeku
Građevinski i arhitektonski fakultet Osijek
Josip Juraj Strossmayer University of Osijek
Faculty of Civil Engineering and Architecture Osijek

Računalno programiranje u građevinarstvu (254810) | Tjedan 3/15
Akademska godina 2025./2026.

Funkcije i modularnost

Definiranje funkcija, argumenti, povratne vrijednosti, moduli, *docstrings*

Marin Grubišić
Izv. prof. dr. sc. / mag. ing. aedif.

Sveučilište J. J. Strossmayera u Osijeku (UNIOS)
Građevinski i arhitektonski fakultet Osijek (GrAFOS)
Zavod za tehničku mehaniku (ZTM)
Predsjednik Katedre za teoriju konstrukcija
Vladimira Preloga 3, **Ured II.26**, HR–31 000 Osijek, Hrvatska

marin.grubisic@gfos.hr | www.maringrubisic.com

Structural &
Earthquake Engineering

Sadržaj prezentacije

1 Uvod u funkcije

2 Naprednije mogućnosti funkcija

3 Moduli

4 GenAI radionica

5 Praktični primjeri

6 Zaključci

Zašto koristiti funkcije?

Uvod u funkcije — motivacija

Do sada smo pisali kod **linearno**, redak po redak. Što se dogodi kada trebamo isti proračun napraviti više puta za različite ulazne veličine?

Bez funkcija — loša praksa:

- Ponavljanje istog koda na više mjesta
- Svaka izmjena zahtijeva promjenu na svim mjestima
- Teško čitanje i održavanje
- Visok rizik od grešaka

S funkcijama — dobra praksa:

- Kod se piše **jednom**, poziva **višestruko**
- Promjena na jednom mjestu vrijedi svugdje
- Jasan, čitljiv i modularan kod
- Lako testiranje i otklanjanje grešaka

> Temeljno načelo: **jedna funkcija = jedan dobro definiran zadatak.** Funkcija treba biti dobro dokumentirana i dovoljno jasno nazvana da njeno ime objašnjava što funkcija radi.

Definiranje i pozivanje funkcije

Uvod u funkcije — sintaksa `def`

Funkcija se definira ključnom riječju **`def`**, nakon koje slijedi naziv funkcije, zagrada s argumentima i dvotočka:

Algoritam 1.1

```python
# Definicija funkcije
def moment_otpora(b, h):
    """Vraća moment otpora pravokutnog presjeka [m³]."""
    Wy = (b * h**2) / 6
    return Wy

# Pozivanje funkcije
Wy1 = moment_otpora(0.30, 0.50)   # b=0.30 m, h=0.50 m
Wy2 = moment_otpora(0.25, 0.45)
Wy3 = moment_otpora(0.40, 0.70)

print(f"Wy1 = {Wy1 * 1e6:.1f} cm³")   # Wy1 = 1250.0 cm³
print(f"Wy2 = {Wy2 * 1e6:.1f} cm³")
print(f"Wy3 = {Wy3 * 1e6:.1f} cm³")
```

- **`def naziv_funkcije(argumenti):`** — zaglavlje s uvlačenjem bloka
- Tijelo funkcije mora biti uvučeno za **4 razmaka**
- **`return`** — vraća rezultat pozivaču (bez **`return`** funkcija vraća **`None`**)

Argumenti funkcije

Uvod u funkcije — argumenti i parametri

Parametri su nazivi u definiciji funkcije; **argumenti** su stvarne vrijednosti pri pozivu:

Algoritam 1.2

```python
# Pozicijski argumenti --- redoslijed je bitan!
def naprezanje_savijanje(M_Ed, b, h):
    """Naprezanje od savijanja u pravokutnom presjeku [MPa]."""
    Wy    = (b * h**2) / 6
    sigma = (M_Ed / Wy) / 1000   # M [kNm] -> sigma [MPa]
    return sigma

sigma1 = naprezanje_savijanje(150.0, 0.30, 0.50) # Pozicionalni!

# Poziv s imenovanim argumentima (keyword arguments)
sigma2 = naprezanje_savijanje(M_Ed=150.0, b=0.30, h=0.50)

# Kod imenovanog poziva redoslijed NE mora biti isti
sigma3 = naprezanje_savijanje(h=0.50, b=0.30, M_Ed=150.0)

print(f"sigma = {sigma1:.2f} MPa")   # sva tri daju isti rezultat
```

> Preporuča se korištenje **imenovanog poziva** (keyword arguments) za funkcije s više od 2–3 argumenta — čitljiviji je i manje podložan greškama zamjene redoslijeda.

Zadane vrijednosti argumenata

Uvod u funkcije — zadane (default) vrijednosti

Argumentima možemo dodijeliti **zadanu (default) vrijednost** koja se koristi ako argument nije naveden pri pozivu:

Algoritam 1.3

```python
import math

def svojstva_kružnog_presjeka(D, jedinice="m"):
    """Geometrijska svojstva punog kružnog presjeka.
    D        : promjer [m]
    jedinice: 'm' ili 'cm' (zadano: 'm')
    """
    A  = math.pi * D**2 / 4
    I  = math.pi * D**4 / 64
    W  = math.pi * D**3 / 32
    i  = D / 4

    if jedinice == "cm":
        return A*1e4, I*1e8, W*1e6, i*1e2   # pretvorba u cm
    return A, I, W, i                        # ostaje u metrima
```

Zadane vrijednosti argumenata

Uvod u funkcije — zadane (default) vrijednosti

Argumentima možemo dodijeliti **zadanu vrijednost** koja se koristi ako argument nije naveden pri pozivu:

Algoritam 1.4

```
# Poziv bez navođenja 'jedinice' --- koristi se zadana vrijednost "m"
A, I, W, i = svojstva_kružnog_presjeka(0.40)
print(f"A={A:.4f} m², I={I:.6f} m⁴")

# Eksplicitni poziv s cm
A, I, W, i = svojstva_kružnog_presjeka(0.40, jedinice="cm")
print(f"A={A:.2f} cm², I={I:.2f} cm⁴")
```

Višestruke povratne vrijednosti

Uvod u funkcije — `return` više vrijednosti

Python funkcija može vratiti **više vrijednosti odjednom** kao n-torku:

Algoritam 1.5

```python
def geometrijska_svojstva_pravokutnika(b, h):
    """Geometrijska svojstva pravokutnog presjeka.
    Ulaz : b [m], h [m]
    Izlaz: A [m²], I [m⁴], W [m³], i [m]
    """
    A = b * h
    I = (b * h**3) / 12
    W = (b * h**2) / 6
    i = (I / A)**0.5   # polumjer tromosti
    return A, I, W, i  # vraća n-torku

# Raspakiranje povratnih vrijednosti
A, I, W, i = geometrijska_svojstva_pravokutnika(0.30, 0.55)

print(f"A = {A*1e4:.1f} cm²")
print(f"I = {I*1e8:.1f} cm⁴")
print(f"W = {W*1e6:.1f} cm³")
print(f"i = {i*1e2:.2f} cm")

# Ili kao n-torka (bez raspakiravanja)
svojstva = geometrijska_svojstva_pravokutnika(0.30, 0.55)
print(f"I = {svojstva[1]*1e8:.1f} cm⁴")
```

Doseg varijabli — lokalne i globalne varijable

Naprednije mogućnosti — doseg varijabli (*scope*)

Varijable definirane **unutar** funkcije su **lokalne** — ne postoje izvan nje:

Algoritam 2.1

```
fcd = 20.0   # GLOBALNA varijabla (definirana izvan funkcije)

def provjeri_presjek(M_Ed, b, h):
    # fcd je dostupna (globalna), ali je bolja praksa proslijediti je
    Wy    = (b * h**2) / 6       # LOKALNA varijabla
    sigma = (M_Ed / Wy) / 1000   # LOKALNA varijabla
    ok    = sigma <= fcd          # koristi globalnu fcd
    return sigma, ok

sigma, ok = provjeri_presjek(150.0, 0.30, 0.50)
print(sigma, ok)

# Wy i sigma iz funkcije NISU dostupni ovdje:
# print(Wy)    # NameError: name 'Wy' is not defined
```

Dobra praksa: sve potrebne vrijednosti **proslijediti kao argumente** umjesto oslanjanja na globalne varijable. To čini funkciju neovisnom, prenosivom i lakšom za testiranje.

Globalne varijable unutar funkcija — `global`

Naprednije mogućnosti — ključna riječ `global`

Po zadanom, dodjela unutar funkcije **uvijek stvara lokalnu varijablu**. Ključna riječ `global` govori Pythonu da koristimo (i mijenjamo) varijablu iz **globalnog dosega**:

Algoritam 2.2

```python
broj_provjera = 0   # globalna varijabla (brojač poziva)

def provjeri_naprezanje(sigma, sigma_Rd):
    """Provjera naprezanja i inkrementiranje globalnog brojača."""
    global broj_provjera       # DEKLARACIJA: koristimo globalnu!
    broj_provjera += 1         # sada smijemo mijenjati globalnu
    return sigma <= sigma_Rd

provjeri_naprezanje(14.2, 20.0)   # poziv 1
provjeri_naprezanje(21.5, 20.0)   # poziv 2
provjeri_naprezanje( 9.8, 20.0)   # poziv 3

print(f"Ukupno provjera: {broj_provjera}")   # → 3
```

Bez `global`: Python bi stvorio novu lokalnu varijablu `broj_provjera` i bacio bi `UnboundLocalError` pri pokušaju `+=` jer lokalna još nije inicijalizirana.

Preporuka: koristite `global` samo za legitimne globalne stanje (npr. brojači, zastavice). Za sve ostalo — **proslijedite vrijednost kao argument i vratite je kroz `return`**.

Dokumentacijski nizovi — *docstrings*

Naprednije mogućnosti — dokumentacija funkcija

Docstring je tekstualni opis funkcije smješten odmah ispod zaglavlja između trostrukih navodnika `"""`. Služi kao ugrađena dokumentacija:

Algoritam 2.3

```python
def progib_proste_grede(q, L, E, I):
    """
    Izračunava maksimalni progib proste grede pod jednolikim
    opterećenjem prema formuli: w = 5*q*L^4 / (384*E*I).

    Argumenti:
        q : jednoliko opterećenje [kN/m]
        L : raspon grede [m]
        E : modul elastičnosti [kN/m²]
        I : moment tromosti [m⁴]

    Vraća:
        w_max : maksimalni progib na sredini raspona [m]
    """
    w_max = (5 * q * L**4) / (384 * E * I)
    return w_max

# Pristup docstringu
help(progib_proste_grede)         # ispis u konzoli
print(progib_proste_grede.__doc__) # izravni pristup
```

Dokumentacijski nizovi — *docstrings*

Naprednije mogućnosti — dokumentacija funkcija

Docstring je tekstualni opis funkcije smješten odmah ispod zaglavlja između trostrukih navodnika `"""`. Služi kao ugrađena dokumentacija:

Algoritam 2.4

```python
def progib_proste_grede(q, L, E, I):
    """
    Izračunava maksimalni progib proste grede pod jednolikim
    opterećenjem prema formuli: w = 5*q*L^4 / (384*E*I).

    Argumenti:
        q : jednoliko opterećenje [kN/m]
        L : raspon grede [m]
        E : modul elastičnosti [kN/m²]
        I : moment tromosti [m⁴]

    Vraća:
        w_max : maksimalni progib na sredini raspona [m]
    """
    w_max = (5 * q * L**4) / (384 * E * I)
    return w_max
```

> Uvijek pišite **docstrings** za sve funkcije koje namjeravate koristiti dulje od jednog dana ili dijeliti s kolegama. U Spyderu se docstring prikazuje u prozoru **Help** pritiskom na **Ctrl+I**.

Lambda funkcije

Naprednije mogućnosti — anonimne `lambda` funkcije

Lambda je kratki zapis za **jednolinjsku anonimnu funkciju** — korisna za jednostavne transformacije i sortiranje:

Algoritam 2.5

```python
# Standardna funkcija:
def pretvori_u_cm4(I_m4):
    return I_m4 * 1e8

# Ekvivalentna lambda funkcija:
pretvori_u_cm4 = lambda I_m4: I_m4 * 1e8

print(pretvori_u_cm4(0.003125))  # 312500.0 cm⁴

# Najčešća primjena: sortiranje složenih struktura
grede = [
    {"oznaka": "G1", "L": 6.5, "sigma": 18.2},
    {"oznaka": "G2", "L": 4.0, "sigma": 9.5},
    {"oznaka": "G3", "L": 5.5, "sigma": 14.7},
]

# Sortiraj po naprezanju (uzlazno)
grede_sort = sorted(grede, key=lambda g: g["sigma"])
for g in grede_sort:
    print(f"{g['oznaka']}: sigma = {g['sigma']:.1f} MPa")
```

Što su moduli i kako ih uvozimo?

Moduli — `import` i organizacija koda

Modul je Python datoteka (`.py`) koja sadrži funkcije, varijable i klase koje možemo koristiti u drugim programima. Python dolazi s **bogatom bibliotekom** modula:

Algoritam 3.1

```python
# 1) Uvoz cijelog modula --- pristup kao modul.funkcija
import math
print(math.sqrt(25))     # 5.0
print(math.pi)           # 3.14159...

# 2) Uvoz s aliasom --- kraći naziv
import math as m
print(m.sin(m.pi / 6))   # 0.5

# 3) Uvoz samo određenih funkcija --- direktan pristup
from math import sqrt, pi, sin, cos, log
print(sqrt(144))         # 12.0  (bez prefiksa math.)

# 4) Uvoz svega --- NE preporuča se (zagađuje namespace!)
# from math import *
```

Preporuča se stil `import math` ili `import math as m` — uvijek je jasno odakle funkcija dolazi. Stil `from math import *` izbjegavajte jer može prouzrokovati konflikte između naziva funkcija.

Korisni standardni moduli

Moduli — pregled standardnih modula

Modul	Opis i primjena
`math`	Matematičke funkcije: `sqrt`, `sin`, `cos`, `log`, `pi`, `exp`
`os`	Rad s datotečnim sustavom: putanje, direktoriji, datoteke
`sys`	Parametri i funkcije Python interpretera
`random`	Generiranje pseudoslučajnih brojeva
`statistics`	Osnovna statistika: `mean`, `stdev`, `median`
`csv`	Čitanje i pisanje CSV datoteka
`datetime`	Rad s datumima i vremenskim oznakama

Korisni standardni moduli

Moduli — pregled standardnih modula

Za inženjerstvo najvažniji **vanjski** moduli (nisu u standardnoj biblioteci, ali su uključeni u Anacondi):

Modul	Primjena
`numpy`	Matrice, vektori, linearna algebra
`matplotlib`	Vizualizacija i grafovi
`pandas`	Tablični podaci, Excel/CSV
`scipy`	Numeričke metode, optimizacija, statistika

Kreiranje vlastitog modula

Moduli — vlastiti modul s inženjerskim funkcijama

Vlastiti modul je jednostavno `.py` datoteka s funkcijama. Primjer datoteke `presjeci.py`:

Algoritam 3.2

```python
# ---- datoteka: presjeci.py ----
import math

def pravokutnik(b, h):
    """Geometrijska svojstva pravokutnog presjeka."""
    A = b * h
    I = (b * h**3) / 12
    W = (b * h**2) / 6
    i = (I / A)**0.5
    return {"A": A, "I": I, "W": W, "i": i}

def krug(D):
    """Geometrijska svojstva punog kružnog presjeka."""
    A = math.pi * D**2 / 4
    I = math.pi * D**4 / 64
    W = math.pi * D**3 / 32
    i = D / 4
    return {"A": A, "I": I, "W": W, "i": i}
```

GenAI radionica #1 — generiranje i provjera funkcija

GenAI — *prompt engineering* za inženjerske funkcije

Zadatak: Zamoli GenAI (ChatGPT, Claude, ...) da generiraj Python funkciju za proračun momenta otpornosti pravokutnog armiranobetonskog presjeka prema EC2.

Lošprompt (previše neodređen):

„Napiši mi Python funkciju za beton."

Dobar prompt (specifičan, inženjerski):

„Napiši Python funkciju `moment_nosivosti_ab(b, d, fcd, fyd, As)` *koja računa moment nosivosti armiranobetonskog pravokutnog presjeka metodom ravnoteže sila prema EN 1992-1-1. Ulazni parametri su: širina presjeka b [m], statička visina d [m], projektna čvrstoća betona fcd [MPa], projektna čvrstoća čelika fyd [MPa] i površina armature As [m²]. Dodaj docstring i primjer poziva."*

Nakon generiranja koda uvijek provjerite: (1) je li formula ispravna, (2) jesu li jedinice konzistentne, (3) je li rezultat fizikalno smislen za poznati primjer. **AI je vaš kalkulator, ne vaš inženjer!**

Primjer 1: Geometrijska svojstva poprečnih presjeka

Praktični primjeri — armiranobetonske konstrukcije

Algoritam 5.1

```python
import math
def svojstva_pravokutnik(b, h):
    """Geometrijska svojstva pravokutnog AB presjeka [m]."""
    A = b * h
    I = (b * h**3) / 12
    W = (b * h**2) / 6
    i = (I / A)**0.5
    return A, I, W, i

def svojstva_krug(D):
    """Geometrijska svojstva kružnog presjeka (stup) [m]."""
    A = math.pi * D**2 / 4
    I = math.pi * D**4 / 64
    W = math.pi * D**3 / 32
    i = D / 4
    return A, I, W, i

presjeci = [("Pravok. 30×55", "P", 0.30, 0.55), # Poziv za seriju presjeka
            ("Pravok. 40×70", "P", 0.40, 0.70),
            ("Krug  D=40cm",  "K", 0.40,  None)]

for naziv, tip, d1, d2 in presjeci:
    if tip == "P":
        A, I, W, i = svojstva_pravokutnik(d1, d2)
    else:
        A, I, W, i = svojstva_krug(d1)
    print(f"{naziv}: A={A*1e4:.1f}cm² "
          f"I={I*1e8:.1f}cm⁴  W={W*1e6:.1f}cm³")
```

Primjer 2: Proračun čelične grede — IPE profil

Praktični primjeri — čelične konstrukcije

Algoritam 5.2

```python
def nosivost_IPE(Wy_cm3, fyk=355.0, gamma_M0=1.0):
    """
    Moment nosivosti čeličnog IPE profila prema EN 1993-1-1.
    Wy_cm3  : moment otpora [cm³] (iz tablica profila)
    fyk     : karakteristična granica popuštanja [MPa] (zad: S355)
    gamma_M0: parcijalni faktor sigurnosti (zad: 1.0)
    """
    fyd   = fyk / gamma_M0          # [MPa] projektna čvrstoća
    Wy_m3 = Wy_cm3 * 1e-6           # pretvorba cm³ -> m³
    Mc_Rd = Wy_m3 * fyd * 1000      # [kNm] moment nosivosti
    return Mc_Rd

# IPE profili: (oznaka, Wy [cm³], G [kg/m])
profili_IPE = [
    ("IPE 300", 557.0,  42.2),
    ("IPE 360", 903.0,  57.1),
    ("IPE 400", 1307.0, 66.3),
    ("IPE 450", 1702.0, 77.6),
]

print(f"{'Profil':>10} | {'Wy [cm³]':>10} | "
      f"{'Mc,Rd [kNm]':>12} | {'G [kg/m]':>9}")
print("-" * 50)
for naziv, Wy, G in profili_IPE:
    Mc = nosivost_IPE(Wy)
    print(f"{naziv:>10} | {Wy:>10.1f} | {Mc:>12.1f} | {G:>9.1f}")
```

Primjer 3: Nosivost drvene grede

Praktični primjeri — drvene konstrukcije

Algoritam 5.3

```python
def nosivost_drvena_greda(b, h, fm_k, kmod=0.8,
                          gamma_M=1.3, kh=1.0):
    """
    Moment nosivosti drvene grede prema EN 1995-1-1 (EC5).
    b, h   : dimenzije presjeka [m]
    fm_k   : karakteristična čvrstoća na savijanje [MPa]
    kmod   : modifikacijski faktor (klasa trajanja, vlažnost)
    gamma_M: parcijalni faktor (zadano: 1.3 za rezano drvo)
    kh     : faktor visine (zadano: 1.0)
    """
    fm_d = kmod * kh * fm_k / gamma_M   # [MPa] projektna čvrstoća
    W    = (b * h**2) / 6               # [m³] moment otpora
    Md_R = W * fm_d * 1000              # [kNm] moment nosivosti
    return Md_R, fm_d

# Klasa čvrstoće C24: fm_k = 24 MPa
razredi = [("C16", 16.0), ("C24", 24.0), ("C30", 30.0)]
b, h    = 0.10, 0.20   # [m]

print(f"Presjek: b={b*100:.0f} cm × h={h*100:.0f} cm")
print(f"{'Klasa':>6} | {'fm,k [MPa]':>10} | "
      f"{'fm,d [MPa]':>10} | {'Md,R [kNm]':>10}")
print("-" * 46)
for klasa, fm_k in razredi:
    Md_R, fm_d = nosivost_drvena_greda(b, h, fm_k)
    print(f"{klasa:>6} | {fm_k:>10.1f} | "
          f"{fm_d:>10.2f} | {Md_R:>10.3f}")
```

Primjer 4: Nosivost zida na vertikalno opterećenje

Praktični primjeri — zidane konstrukcije

Algoritam 5.4

```python
def nosivost_zida(t, L, fk, gamma_M=2.5, Phi=0.8):
    """
    Vertikalna nosivost nearmirane zidane ploče prema EN 1996-1-1.
    t      : debljina zida [m]
    L      : duljina zida [m]
    fk     : karakteristična tlačna čvrstoća zida [MPa]
    gamma_M: parcijalni faktor sigurnosti (zad: 2.5)
    Phi    : faktor smanjenja kapaciteta (ekscentricitet)
    """
    fd   = fk / gamma_M              # [MPa] projektna čvrstoća
    A    = t * L                     # [m²] površina presjeka
    NRd  = Phi * A * fd * 1000       # [kN] nosivost
    return NRd, fd

# Zidovi od opeke različitih klasa
zidovi = [
    ("Opeka M5",  2.5, 0.25, 2.0),
    ("Opeka M10", 5.0, 0.25, 2.0),
    ("Blok M5",   3.5, 0.20, 2.5),
]

print(f"{'Materijal':>12} | {'fk[MPa]':>8} | "
      f"{'t[m]':>5} | {'L[m]':>5} | {'NRd[kN]':>9}")
print("-" * 50)
for naziv, fk, t, L in zidovi:
    NRd, fd = nosivost_zida(t, L, fk)
    print(f"{naziv:>12} | {fk:>8.1f} | "
          f"{t:>5.2f} | {L:>5.1f} | {NRd:>9.1f}")
```

Primjer 5: Knjižnica funkcija za proračun grede

Praktični primjeri — modularna organizacija proračuna

Algoritam 5.5

```python
def moment_tromosti(b, h): # Modularna organizacija proračuna AB grede prema EC2
    return (b * h**3) / 12

def naprezanje(M_Ed, b, h):
    W = (b * h**2) / 6
    return (M_Ed / W) / 1000   # [MPa]

def progib(q, L, E, b, h):
    I     = moment_tromosti(b, h)    # poziv druge funkcije!
    w_max = (5 * q * L**4) / (384 * E * I)
    return w_max

def provjera_grede(naziv, L, b, h, q, fcd=20.0, Ecm=32e6):
    """Kompletna provjera nosivosti i upotrebljivosti AB grede."""
    M_Ed  = q * L**2 / 8
    sigma = naprezanje(M_Ed, b, h)
    w_max = progib(q, L, Ecm, b, h)
    L250  = L / 250
    ok_N  = sigma <= fcd
    ok_U  = w_max <= L250
    print(f"  Greda: {naziv}")
    print(f"  M_Ed  = {M_Ed:.2f} kNm")
    print(f"  sigma = {sigma:.2f} MPa  --> {'OK' if ok_N else 'NOK'}")
    print(f"  w_max = {w_max*1000:.2f} mm  --> {'OK' if ok_U else 'NOK'}")
    print(f"  L/250 = {L250*1000:.1f} mm")

provjera_grede("G1 (30×55 cm)", L=6.0, b=0.30, h=0.55, q=25.0)
provjera_grede("G2 (25×45 cm)", L=4.5, b=0.25, h=0.45, q=18.0)
```

Zaključci

Pregled najvažnijih aspekata ovog tjedna

Što smo naučili:

- **Definiranje funkcija** (`def`) — temeljni gradivni blok modularnog koda
- **Argumenti i parametri**: pozicijski, imenovani (keyword) i zadane vrijednosti (*default*)
- **Višestruke povratne vrijednosti** (`return a, b, c`) — povrat n-torke i raspakiranje
- **Doseg varijabli** (*scope*) — lokalne vs. globalne varijable
- **Docstrings** — dokumentacija funkcija (vidljiva u Spyder *Help* prozoru)
- **Lambda funkcije** — jednolinijske anonimne funkcije za sortiranje i transformacije
- **Moduli** i načini uvoza (`import`, `from ... import`, alias)
- **Kreiranje vlastitog modula** — organizacija koda u `.py` datoteke
- **GenAI za generiranje funkcija** — korektno promptanje i kritička provjera

Zlatno pravilo: jedna funkcija = jedan zadatak. Funkcija koja radi previše stvari odjednom teža je za razumijevanje, testiranje i ponovnu upotrebu.

Sljedeći tjedan (4/15): NumPy — vektori, matrice i rješavanje sustava linearnih jednadžbi.

Domaća zadaća #3

Rok predaje: do početka sljedećeg termina nastave

Zadatak: Napišite Python **modul** (datoteku `presjeci_dz3.py`) koji sadrži skup funkcija za proračun geometrijskih svojstava triju vrsta poprečnih presjeka:

1. Funkciju `pravokutni_presjek(b, h)` koja vraća rječnik s ključevima `"A"`, `"I"`, `"W"`, `"i"` za pravokutni presjek
2. Funkciju `kružni_presjek(D)` koja na isti način obrađuje puni kružni presjek
3. Funkciju `šuplji_kružni_presjek(D, t)` koja obrađuje cijev vanjskog promjera D i debljine stijenke t: $I = \dfrac{\pi\left(D^4 - d^4\right)}{64}, \quad d = D - 2t$
4. Funkciju `ispiši_svojstva(naziv, svojstva)` koja lijepo formatira i ispisuje rezultate iz rječnika (uz pretvorbu u cm, cm^2, cm^4, cm^3)

Zahtjevi:

- Sve funkcije moraju imati **docstring** s opisom argumenta i povratnih vrijednosti
- U zasebnoj datoteci `glavni_dz3.py` uvezi modul i pozovi sve 3 funkcije za presjeke po vlastitom izboru dimenzija
- Dodajte zadane vrijednosti (*default*) tamo gdje to ima smisla

Hvala na pozornosti!

Pitanja?

Marin Grubišić
Izv. prof. dr. sc. / mag. ing. aedif.

Sveučilište J. J. Strossmayera u Osijeku (UNIOS)
Građevinski i arhitektonski fakultet Osijek (GrAFOS)
Zavod za tehničku mehaniku (ZTM)
Predsjednik Katedre za teoriju konstrukcija
Vladimira Preloga 3, **Ured II.26**, HR–31 000 Osijek, Hrvatska

marin.grubisic@gfos.hr | www.maringrubisic.com

Structural &
Earthquake Engineering